

PyTorch Autograd

Computational Graphs, Gradients, and Backpropagation

Abstract

Autograd is PyTorch’s automatic differentiation engine. It records tensor operations as a dynamic computational graph, then applies the chain rule in reverse to compute derivatives. These notes develop the idea from elementary scalar derivatives to a one-neuron classifier, vector-valued inputs, gradient accumulation, and deliberate control of graph construction. Every code block is an equivalent explanatory reconstruction based on the demonstrated computations; it is not presented as a verbatim notebook transcription.

Contents

1	Why automatic differentiation is needed	2
1.1	A derivative that is easy to code by hand	2
1.2	Nesting introduces the chain rule	2
2	Autograd as reverse-mode automatic differentiation	3
2.1	Graph vocabulary	3
3	First scalar Autograd computation	3
4	Nested functions and reverse-mode Autograd	4
5	Leaf and non-leaf tensors	5
6	One-neuron placement model	6
6.1	Manual chain-rule derivatives	6
7	Vector inputs and multivariable gradients	8
8	Gradient accumulation and clearing	9
9	Controlling graph construction	10
9.1	Change a leaf’s requirement in place	10
9.2	Detach a tensor from its history	11
9.3	Temporarily disable recording with a context manager	11
10	From Autograd to loss minimization	11
11	Practical observations and pitfalls	12

1 Why automatic differentiation is needed

1.1 A derivative that is easy to code by hand

Consider

$$y = x^2. \quad (1)$$

Here, x is the scalar input and y is the scalar output. Differentiating gives

$$\frac{dy}{dx} = 2x. \quad (2)$$

The derivative measures the instantaneous sensitivity of y to a small change in x . At $x = 2$ it is 4; at $x = 3$ it is 6. The formula matters because gradient-based learning uses exactly such sensitivities to decide how model parameters should change.

Listing 1: hand-coded derivative of a square.

```
def dy_dx(x):
    return 2 * x

print(dy_dx(2.0)) # 4.0
print(dy_dx(3.0)) # 6.0
```

The function contains a derivative formula derived in advance. This is manageable for one operation but does not scale to a deep composition of functions.

1.2 Nesting introduces the chain rule

Now define

$$y = x^2, \quad (3)$$

$$z = \sin(y). \quad (4)$$

The symbols x , y , and z denote the input, intermediate value, and final output. The two local derivatives are

$$\frac{dy}{dx} = 2x, \quad \frac{dz}{dy} = \cos(y). \quad (5)$$

The first local derivative describes the square operation; the second describes the sine operation. The chain rule combines them:

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx} = \cos(y) 2x = 2x \cos(x^2). \quad (6)$$

Thus dz/dx computes the total sensitivity of the final output to the original input. At $x = 3$, it is approximately -5.4668 ; at $x = 4$, it is approximately -7.6613 . Adding more nested operations adds more local derivative factors. Neural networks contain many such compositions, so deriving and coding every path manually becomes impractical.

Listing 2: manual chain-rule implementation.

```
import math

def dz_dx(x):
    y = x ** 2
    return math.cos(y) * (2 * x)

print(dz_dx(3.0)) # approximately -5.4668
print(dz_dx(4.0)) # approximately -7.6613
```

The code first reconstructs the intermediate value y , evaluates the two local factors, and multiplies them. An additional nested function would require another factor and another piece of derivative code.

2 Autograd as reverse-mode automatic differentiation

Autograd provides automatic differentiation for tensor operations. When at least one relevant input has `requires_grad=True`, PyTorch records the forward operations in a directed acyclic graph (DAG). Calling `backward()` traverses this graph from output to leaves and applies the chain rule.

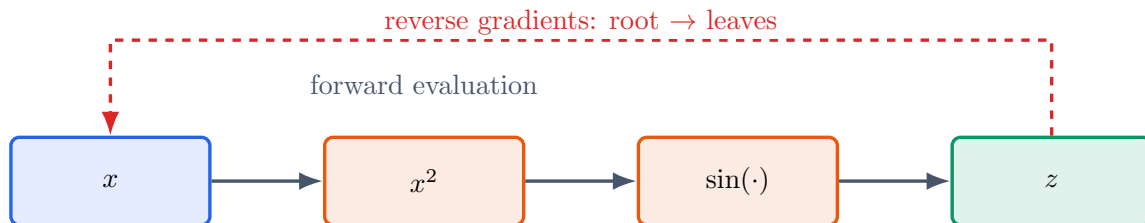


Figure 1: Forward evaluation computes values; reverse traversal propagates gradients. Conceptual reference: [PyTorch Autograd mechanics](#).

Intuition

The forward pass answers “what value does the computation produce?” The backward pass answers “how sensitive is that value to each tracked leaf tensor?” Reverse mode is efficient when a scalar loss depends on many parameters, which is the standard neural-network setting.

2.1 Graph vocabulary

- A **leaf tensor** is typically created directly by the user. Trainable parameters are leaves with `requires_grad=True`; their gradients are stored in `.grad`.
- A **non-leaf tensor** is produced by a tracked operation. It carries history through `grad_fn`.
- An **operation node** knows how to propagate an incoming gradient through its local derivative.
- An **edge** represents data dependence and the route along which a gradient can flow backward.
- The final output is the **root** of the reverse traversal. For training it is usually a scalar loss.

The graph is dynamic: ordinary Python control flow determines which operations are recorded during that execution.

3 First scalar Autograd computation

Let $x = 3$ and $y = x^2$. The forward result is $y = 9$, while the expected derivative is $dy/dx = 2x = 6$.

Listing 3: square, graph history, and backward propagation.

```
import torch

x = torch.tensor(3.0, requires_grad=True)  # tracked leaf scalar
y = x ** 2                                # non-leaf scalar

print(x)      # tensor(3., requires_grad=True)
print(y)      # tensor(9., grad_fn=<PowBackward0...>)
print(x.is_leaf)  # True
print(y.is_leaf)  # False
print(y.grad_fn)  # backward function for the power operation

y.backward()
print(x.grad)     # tensor(6.)
```

The first line creates a zero-dimensional floating-point tensor. `requires_grad=True` asks PyTorch to track subsequent differentiable operations involving x . The power operation creates y and attaches a backward-function object to it. Because y is scalar, `y.backward()` implicitly seeds the reverse pass with $dy/dy = 1$. The resulting derivative is accumulated in `x.grad`.

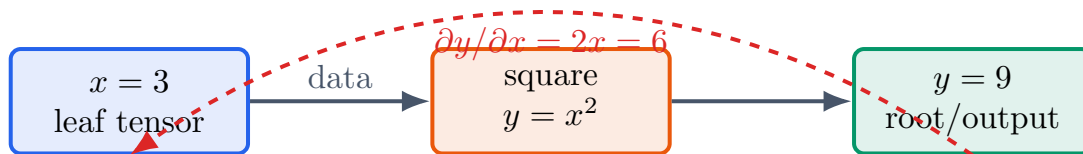


Figure 2: Computational graph for $y = x^2$ at $x = 3$. Reference: [PyTorch automatic differentiation tutorial](#).

Pitfall

Only floating-point or complex tensors can require gradients. Writing `torch.tensor(3, requires_grad=True)` uses an integer dtype and raises an error. Use 3.0 or specify a floating dtype.

4 Nested functions and reverse-mode Autograd

For $y = x^2$ and $z = \sin(y)$, PyTorch stores both operations. Backward propagation begins from z and multiplies the sine derivative by the square derivative.

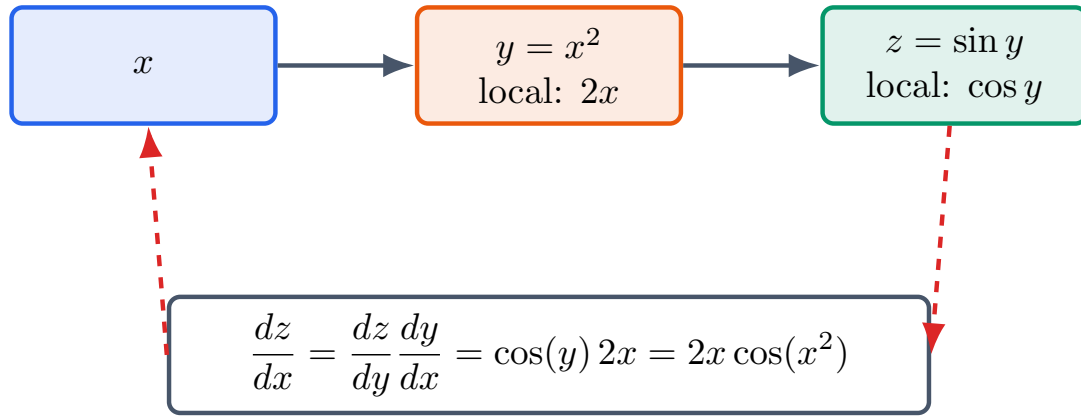


Figure 3: Local derivatives combine through the chain rule. Conceptual reference: [Autograd mechanics](#).

Listing 4: nested square and sine computation.

```
def nested_gradient(value):
    x = torch.tensor(float(value), requires_grad=True)
    y = x ** 2                # non-leaf; y.grad_fn is PowBackward
    z = torch.sin(y)          # non-leaf root; SinBackward
    z.backward()              # dz/dx = cos(x^2) * 2x
    return y.item(), z.item(), x.grad.item()

print(nested_gradient(3.0))
# (9.0, approximately 0.4121, approximately -5.4668)

print(nested_gradient(4.0))
# (16.0, approximately -0.2879, approximately -7.6613)
```

Each call creates a fresh graph. For $x = 4$, $y = 16$, $z = \sin(16) \approx -0.2879$, and $dz/dx = 8 \cos(16) \approx -7.6613$. The agreement with the hand calculation verifies that Autograd is applying the same chain rule automatically.

5 Leaf and non-leaf tensors

In the nested example, x is a leaf because it was created directly. The intermediate y is non-leaf because it resulted from a tracked operation. PyTorch normally populates `.grad` only on tracked leaves, even though intermediate gradients are computed transiently during backpropagation.

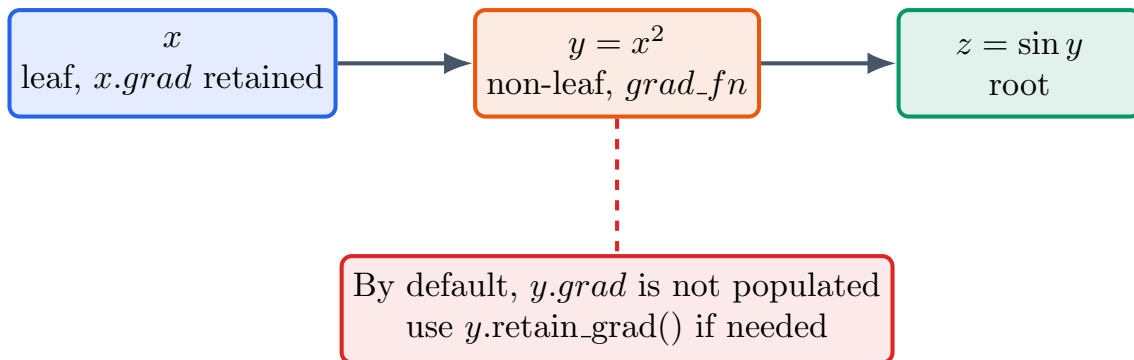


Figure 4: Leaves retain gradients by default; intermediate tensors retain history but not normally their `.grad` field. Reference: [PyTorch Autograd notes](#).

Listing 5: observing and retaining a non-leaf gradient.

```
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
z = torch.sin(y)

y.retain_grad() # request storage for this non-leaf tensor
z.backward()

print(x.grad) # dz/dx = 2x*cos(x^2), approximately -5.4668
print(y.grad) # dz/dy = cos(y), approximately -0.9111
```

Without `y.retain_grad()`, accessing `y.grad` produces `None` and usually a warning. Retaining intermediate gradients is useful for debugging, but unnecessary retention consumes memory.

6 One-neuron placement model

A small binary classifier makes the connection to neural-network training explicit. Let $x = 6.7$ be a student's CGPA, $y = 0$ the placement label, $w = 1$ the weight, and $b = 0$ the bias. The forward equations are

$$z = wx + b, \quad (7)$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad (8)$$

$$L = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]. \quad (9)$$

Here, z is the logit, $\hat{y}$ is the predicted probability of placement, and L is binary cross-entropy. The sigmoid maps any real logit to $(0, 1)$. The loss is small for a correct confident probability and large for an incorrect confident probability. With the chosen values, $z = 6.7$, $\hat{y} \approx 0.9987706$, and $L \approx 6.7012302$. The loss is high because the model predicts placement with high confidence while the target is zero.

6.1 Manual chain-rule derivatives

The local derivatives are

$$\frac{\partial L}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}, \quad (10)$$

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}), \quad (11)$$

$$\frac{\partial z}{\partial w} = x, \quad \frac{\partial z}{\partial b} = 1. \quad (12)$$

The first equation describes how the loss changes with predicted probability. The second is the local slope of sigmoid. The last two measure the logit's sensitivity to the parameters. Multiplication cancels the sigmoid terms:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w} = (\hat{y} - y)x, \quad (13)$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial b} = \hat{y} - y. \quad (14)$$

Thus $\partial L / \partial w \approx 6.6917630$ and $\partial L / \partial b \approx 0.9987706$. A positive derivative means decreasing the parameter will reduce the loss for a sufficiently small positive learning rate.

Listing 6: equivalent reconstruction of the manual forward pass.

```
import math

x, target = 6.7, 0.0
w, b = 1.0, 0.0
z = w * x + b
prediction = 1.0 / (1.0 + math.exp(-z))
loss = -(target * math.log(prediction)
          + (1.0 - target) * math.log(1.0 - prediction))

print(z)           # 6.7
print(prediction)  # approximately 0.9987706
print(loss)        # approximately 6.7012302
```

The first block initializes one observation and two parameters. The next three lines implement the affine transform, sigmoid, and binary cross-entropy exactly in the order represented by the computational graph.

Listing 7: equivalent reconstruction of the manual backward pass.

```
dL_dpred = (prediction - target) / (
    prediction * (1.0 - prediction)
)
dpred_dz = prediction * (1.0 - prediction)
dz_dw = x
dz_db = 1.0

dL_dw = dL_dpred * dpred_dz * dz_dw
dL_db = dL_dpred * dpred_dz * dz_db
print(dL_dw)  # approximately 6.6917630
print(dL_db)  # approximately 0.9987706
```

The four local derivatives expose the bookkeeping that Autograd removes. The two products trace the paths from L back to w and b .

Listing 8: the same model differentiated by Autograd.

```
import torch.nn.functional as F

x = torch.tensor(6.7)           # data: no gradient needed
target = torch.tensor(0.0)      # label: no gradient needed
w = torch.tensor(1.0, requires_grad=True) # trainable leaf
b = torch.tensor(0.0, requires_grad=True) # trainable leaf

z = w * x + b
prediction = torch.sigmoid(z)
loss = F.binary_cross_entropy(prediction, target)
loss.backward()

print(prediction) # tensor(0.9988, grad_fn=<SigmoidBackward...>)
print(loss)      # tensor(6.7012, grad_fn=<BinaryCrossEntropyBackward...>)
print(w.grad)    # tensor(6.6918)
print(b.grad)    # tensor(0.9988)
```

Only w and b require gradients because they are the quantities to optimize. A single call to `loss.backward()` computes both parameter derivatives.

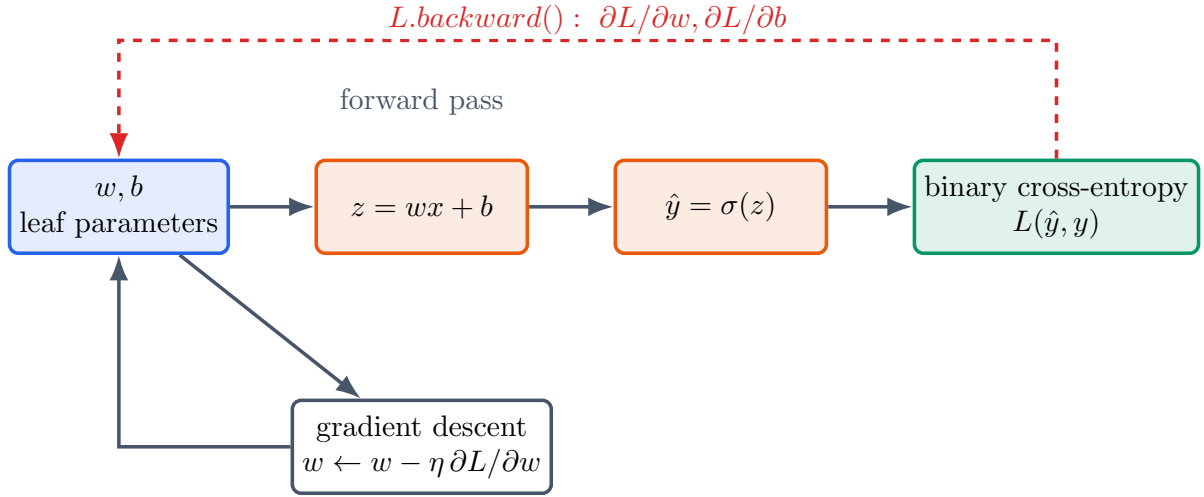


Figure 5: A scalar loss connects forward prediction to reverse-mode parameter gradients and gradient descent. Reference: [PyTorch optimization tutorial](#).

7 Vector inputs and multivariable gradients

Let

$$\mathbf{x} = [x_1, x_2, x_3] = [1, 2, 3] \quad (15)$$

and define

$$y = \text{mean}(\mathbf{x}^2) = \frac{x_1^2 + x_2^2 + x_3^2}{3}. \quad (16)$$

The square is elementwise, and the mean reduces three values to one scalar. Therefore $y = 14/3 \approx 4.6667$. Each partial derivative is

$$\frac{\partial y}{\partial x_i} = \frac{2x_i}{3}, \quad i \in \{1, 2, 3\}. \quad (17)$$

The index i identifies one input component. This formula computes how the scalar mean changes if only that component changes. Evaluating it gives the gradient vector

$$\nabla_{\mathbf{x}} y = \left[\frac{2}{3}, \frac{4}{3}, 2 \right]. \quad (18)$$

This vector collects all partial derivatives and generalizes the scalar derivative used earlier.

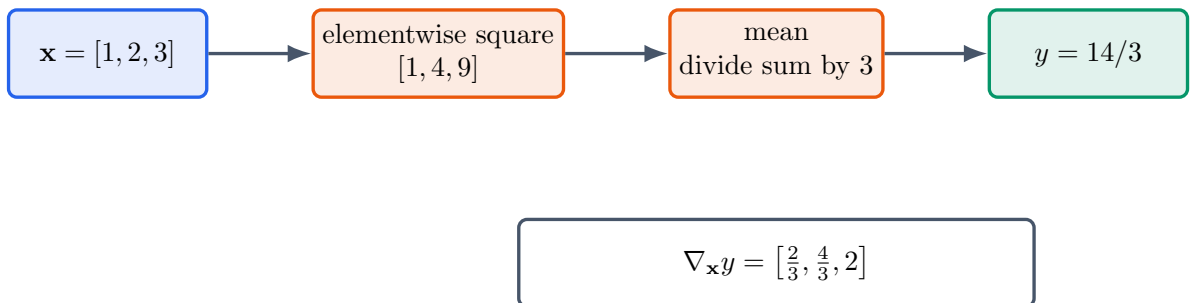


Figure 6: Elementwise square followed by mean reduction creates one scalar suitable for an unseeded `backward()` call. Reference: [PyTorch Autograd tutorial](#).

Listing 9: gradient of mean squared components.

```
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = (x ** 2).mean() # scalar: (1 + 4 + 9) / 3

y.backward()
print(y) # tensor(4.6667, grad_fn=<MeanBackward0...>)
print(x.grad) # tensor([0.6667, 1.3333, 2.0000])
```

The vector itself is one leaf tensor. Its `.grad` field has the same shape and stores one derivative per element.

Pitfall

Calling `backward()` with no argument is directly valid only for a scalar output. If an output tensor `u` has several elements, provide an upstream vector `v` by calling `u.backward(v)`; PyTorch then computes the vector–Jacobian product $\mathbf{v}^\top J$. Alternatively, reduce the output to a scalar such as a sum or mean. This distinction is a general Autograd rule; the demonstrated vector example deliberately ends in a scalar mean.

8 Gradient accumulation and clearing

PyTorch *adds* new gradients into an existing leaf’s `.grad` field. It does not overwrite them. For $x = 2$ and $y = x^2$, one pass contributes 4. A second fresh forward/backward pass contributes another 4, giving 8 unless the old value is cleared.

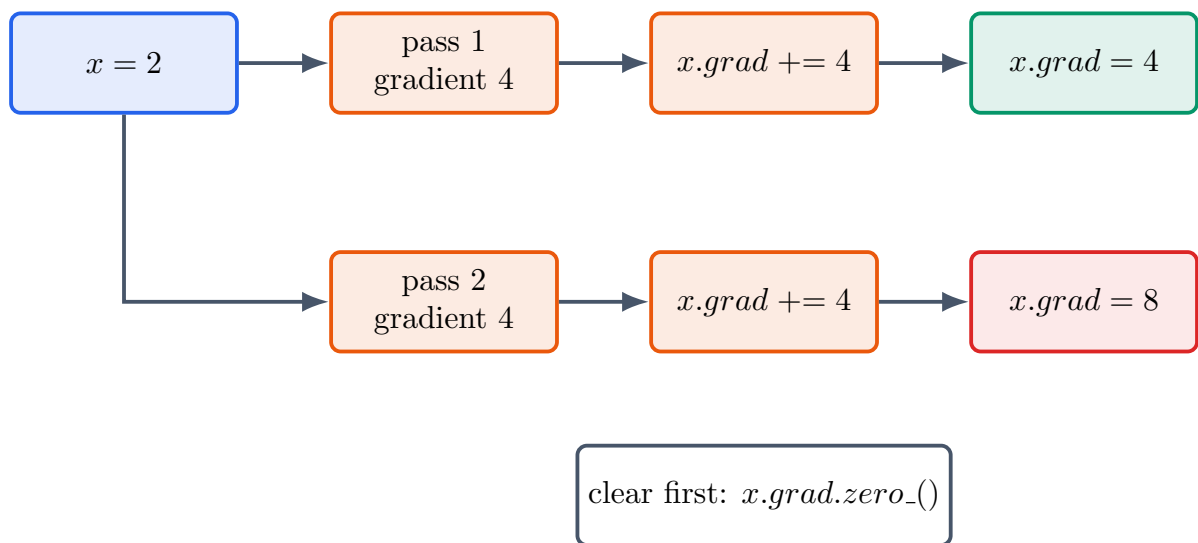


Figure 7: Accumulation is intentional: contributions are added until gradients are explicitly cleared. Reference: [PyTorch recipe for zeroing gradients](#).

Listing 10: accumulation and in-place clearing.

```
x = torch.tensor(2.0, requires_grad=True)

y = x ** 2
y.backward()
print(x.grad)  # tensor(4.)

# A fresh forward graph contributes another gradient.
y = x ** 2
y.backward()
print(x.grad)  # tensor(8.) = 4 + 4

x.grad.zero_()  # in-place reset of the gradient tensor
y = x ** 2
y.backward()
print(x.grad)  # tensor(4.) again
```

The underscore in `zero_()` denotes an in-place operation. In a model training loop, the usual choice is `optimizer.zero_grad()` before the next backward pass. Accumulation is useful when intentionally combining several mini-batches, but accidental accumulation produces incorrect parameter updates.

Pitfall

A second call to `backward()` on the *same* graph normally fails because saved intermediates are freed after the first backward pass. The reconstruction above recomputes y and therefore creates a fresh graph, matching the repeated-pass demonstration. Use `retain_graph=True` only when reuse is genuinely required, because retaining a graph costs memory.

9 Controlling graph construction

Gradient tracking is needed during training but usually not during inference. Disabling it saves memory and avoids unnecessary backward metadata. Three mechanisms address different scopes.

9.1 Change a leaf's requirement in place

Listing 11: disabling tracking on a leaf tensor.

```
x = torch.tensor(2.0, requires_grad=True)
x.requires_grad_(False)  # in-place state change

y = x ** 2
print(x.requires_grad)  # False
print(y.grad_fn)        # None
```

The method changes whether future operations involving this leaf are tracked. It is appropriate when a parameter is deliberately frozen or a tensor no longer needs derivatives.

9.2 Detach a tensor from its history

Listing 12: creating a graph-disconnected tensor.

```
x = torch.tensor(2.0, requires_grad=True)
z = x.detach()

y_tracked = x ** 2
u_untracked = z ** 2

print(z)                # tensor(2.)
print(z.requires_grad)   # False
print(y_tracked.grad_fn) # power backward function
print(u_untracked.grad_fn) # None
```

`detach()` returns a tensor with the same values that is disconnected from the current graph. It generally shares underlying storage with the source, so unsafe in-place mutation of either view can have side effects. Use it to stop a gradient path while continuing ordinary tensor computation.

9.3 Temporarily disable recording with a context manager

Listing 13: inference-like computation under no-grad.

```
x = torch.tensor(2.0, requires_grad=True)

with torch.no_grad():
    y = x ** 2

print(y)                # tensor(4.)
print(y.requires_grad)   # False
print(y.grad_fn)         # None
```

`torch.no_grad()` disables graph recording only inside its block. It is convenient for validation and inference because model parameters can remain trainable outside the block.

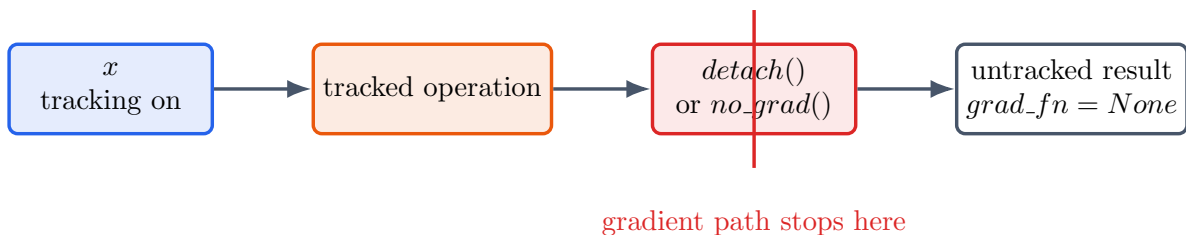


Figure 8: Detachment and no-gradient scopes create explicit graph boundaries. Reference: [PyTorch no_grad documentation](#).

Key takeaway

`requires_grad_(False)` changes a tensor's future tracking state; `detach()` returns a disconnected tensor; `no_grad()` temporarily suppresses recording for all operations in a block.

10 From Autograd to loss minimization

A complete learning step has four conceptual stages:

1. **Forward pass:** compute predictions from data and current parameters.
2. **Loss computation:** compare predictions with targets.
3. **Backward pass:** call `loss.backward()` to compute parameter gradients.
4. **Update and clear:** move parameters opposite the gradient, then clear accumulated gradients.

For a parameter w , gradient descent uses

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}. \quad (19)$$

Here, L is the scalar loss, $\partial L / \partial w$ is its sensitivity to the parameter, and $\eta > 0$ is the learning rate. The subtraction moves w locally toward lower loss; η controls the step size. This equation matters because Autograd supplies the derivative while the optimizer applies the update.

Listing 14: minimal optimization loop connecting all concepts.

```
x = torch.tensor(6.7)
target = torch.tensor(0.0)
w = torch.tensor(1.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)
learning_rate = 0.1

for step in range(3):
    prediction = torch.sigmoid(w * x + b)          # forward pass
    loss = F.binary_cross_entropy(prediction, target)

    loss.backward()                                # populate w.grad, b.grad

    with torch.no_grad():                          # update is not part of graph
        w -= learning_rate * w.grad
        b -= learning_rate * b.grad

    w.grad.zero_()                                # prevent accumulation
    b.grad.zero_()
```

The data and label do not require gradients. The parameters do. The update is wrapped in `no_grad()` because differentiating the optimizer step is unnecessary in ordinary training. Clearing gradients prepares the leaves for the next iteration.

11 Practical observations and pitfalls

- **History follows operations, not variable names.** Reassigning a Python name does not alter the graph already attached to a tensor.
- **In-place operations require care.** PyTorch may reject an in-place change when a saved value is needed for a correct backward pass. Gradient clearing is a safe, deliberate in-place operation on the gradient buffer after it has been consumed.
- **Gradients are local sensitivities.** A large magnitude means the loss is highly sensitive to that parameter at the current point; the sign indicates the direction of increase.
- **A graph is rebuilt each forward pass.** This supports data-dependent Python control flow but also explains why repeated iterations create fresh graphs.
- **Non-leaf gradients still participate.** They are computed for chain-rule propagation even when their `.grad` fields are not retained.
- **Inference should not record a graph.** Suppressing recording reduces memory use and avoids meaningless backward bookkeeping.

12 Review questions

1. Why does setting `requires_grad=True` on x cause $y = x^2$ to have a non-null `grad_fn`?

2. Derive $d \sin(x^2)/dx$ by identifying both local derivatives.
3. Why is y non-leaf in the nested example, and how can its gradient be retained for debugging?
4. For $\mathbf{x} = [1, 2, 3]$ and $y = \text{mean}(\mathbf{x}^2)$, why does `x.grad` contain three values?
5. Why does a second backward pass add to an existing gradient rather than replace it?
6. Distinguish `detach()`, `no_grad()`, and `requires_grad_(False)`.
7. In the one-neuron model, why is the loss large when $y = 0$ but $\hat{y} \approx 0.999$?
8. Why must ordinary parameter updates be performed without gradient tracking?

13 Summary

Autograd turns a forward tensor program into a dynamic computational graph. Leaves represent tracked inputs or parameters; non-leaves represent intermediate results and carry backward-function history. Reverse-mode traversal starts at a scalar output, propagates local derivatives, and combines them with the chain rule. Gradients accumulate in leaf buffers, so training loops must clear them deliberately. Graph construction can be stopped for a tensor with `detach()`, changed in place with `requires_grad_()`, or temporarily disabled with `no_grad()`. These mechanisms allow neural-network training to reduce a loss without manually deriving the enormous nested function represented by the model.